Capturing Provenance of Academy Agents with Flowcept

Hands-on tutorial for ATPESC 2024

Amal Gueroudji

ATPESC

Argonne National Laboratory

Provenance — another kind of data

Provenance: another kind of data

Alongside the inputs and results a workflow produces, there is **another kind of data** worth keeping: provenance — the recorded history of *what ran, on what, producing what, when, where, and how it all connects*.

- It is newly **essential** because of AI: agentic workflows are non-deterministic — the same prompt yields different outputs, and the control flow is *decided at runtime*, not written in your source.
- Without that record an AI run is *not reproducible* and its failures are *invisible* — you cannot say what the agent actually did.

In this session we **capture and study provenance with Flowcept**, on agentic workflows built with Academy.

Why capture provenance at all?

Provenance = the recorded history of *what ran, on what inputs, producing what outputs, when, where, and how it all connects*.

- **Reproducibility** — rerun and get the same result; know exactly what code, data, and parameters produced a number.
- **Debugging** — when something breaks, see *which* step failed, on *what* input, and why — not just a stack trace.
- **Trust & audit** — verify and defend results; a scientific claim is only as good as the record behind it.
- **Cost & performance** — see where time and tokens go.

For classic HPC workflows this is valuable. For agentic systems it becomes essential — next slide.

Why it matters more for agentic systems

Traditional code does what you wrote. **Agents decide what to do at runtime** — so the "what happened" isn't in your source.

- **Non-determinism.** LLM calls sample — you can't reproduce a run without the actual prompt, response, model, and tokens.
- **Autonomy & emergence.** `@loops` act on their own; agents choose which tool/peer to call. The control flow is *decided*, not declared — capture is the only record of the real path.
- **Distribution.** Work spans agents, processes, nodes, and *frameworks* (Academy + LangGraph + Parsl); lineage across those boundaries is invisible without provenance.
- **Opaque reasoning.** Prompts, tool calls, and delegations bury the "why." Provenance keeps every prompt/response.
- **Failure is normal.** Small LLMs propose malformed inputs; agents derail mid-plan — you need the exact failing step + input.

Non-deterministic, autonomous, distributed — exactly the case provenance was made for.

The tools — Flowcept + Academy

What is Flowcept?

Flowcept is a runtime **provenance capture** system: it records *what ran, with what inputs/outputs, when, and how it connects* — the provenance graph.

- **Data model**: `campaign` → `workflow` → `task` (action / loop / llm_call), with `used`/`generated` and `parent_task_id` lineage.
- Its *agentic* plugin patches Academy's `Runtime`, so capture is automatic — zero changes to agent logic.
- Runs **offline** here (no Redis / no Mongo): records land in a per-run `flowcept_buffer.jsonl`.

What is Academy?

A framework for **distributed multi-agent systems** for science — these are the agentic workflows we instrument (unchanged).

- `Agent` — a class holding state + behaviors
- `@action` — a callable behavior (request/response)
- `@loop` — autonomous background behavior
- `Handle` — a remote reference to an agent
- `Manager` + `Exchange` — launch agents & route messages

Scales from local threads → processes → Globus Compute / Parsl across nodes.

Academy: the shape of an agent

```
class Coordinator(Agent):
    @action
    async def process(self, text: str) -> str:
        text = await self.lowerer.lower(text)       # cross-agent call
        return await self.reverser.reverse(text)    # cross-agent call

async with await Manager.from_exchange_factory(
        factory=LocalExchangeFactory(),
        executors=ThreadPoolExecutor()) as manager:
    coord = await manager.launch(Coordinator, args=(lo, re))
    await coord.process("DEADBEEF")      # -> "feebdaed"
```

These are Academy's own examples (`github.com/academy-agents/academy`); we instrument them unchanged.

The hands-on session

The hands-on session

Take the **upstream Academy examples**, instrument each with **Flowcept** to capture provenance
(zero changes to agent logic), and **inspect** what happened — including where things fail.

Part	Content
1	Provenance — another kind of data, and why agentic makes it essential
2	The tools: Flowcept + Academy
3	Flowcept + examples 1, 2, 3 — and how to inspect provenance
4	Advanced: agents & LLMs (6, 8), LangGraph + real xTB (7), Parsl (5), failures

Terminal-only (no images): text summaries, ASCII dashboards, a markdown provenance card — runs over SSH on Aurora. Every run writes to its own `runs/<id>_<date-time>/`.

How the tutorial is laid out

Eight upstream Academy examples, each a **step-by-step exercise**. As shipped each one *runs but records nothing*; you turn provenance on one **STEP** at a time by uncommenting a block and re-running.

```
exercises/
  local/     run on a laptop / login shell    (python exercise.py)
  aurora/    run on ALCF Aurora               (qsub submit.pbs; shared env.sh)
    <id>/    exercise.py  solution.py  agent_src.py  README.md  [submit.pbs]
```

- `exercise.py` — STEP 0 baseline, then STEP 1 capture, 2 inspect, 3 analyze, 4 card, 5 query (uncomment as you go)
- `solution.py` — every step already enabled (the reference)
- `flowcept_academy/` is a small **library** the exercises build on

Setup — one command, one env, from scratch

```
bash setup/install.sh          # builds ONE conda env for all 8 (CPU)
conda activate flowcept-academy
cd exercises/local/01-actor-client && python exercise.py
```

`install.sh` builds a single `flowcept-academy` conda env and installs Flowcept (agentic branch) + Academy + Parsl + LangGraph + a CPU build of `torch`/`transformers`, plus `rdkit`/`xtb-python` for example 07's real chemistry, and drops the offline Flowcept settings in place. One env for the whole tutorial — no venv, no per-exercise env. No Redis, no Mongo, no GPU.

Capture + inspect — examples 1, 2, 3

Turn capture on — STEP 1

```
from flowcept_academy.capture import captured    # zero agent-code changes
from flowcept_academy.util import capture_run     # -> runs/<id>_<date-time>/

with capture_run("03-agent-agent") as run_dir:
    with captured(workflow_name="03-agent-agent"):
        result = asyncio.run(run())     # the upstream example, now captured
    # -> flowcept_buffer.jsonl in run_dir  (offline: no Redis / no Mongo)
```

In the exercise you just uncomment this block and re-run. Everything a run produces — buffer, perf CSV, the markdown card — lands together in the run dir:

```
cd exercises/local/03-agent-agent && python exercise.py
# -> runs/03-agent-agent_<date-time>/ : flowcept_buffer.jsonl + card.md + perf
    .csv
```

Outcomes: examples 1, 2, 3

Each example exercises a different slice of the provenance model:

Example	Provenance captured
01 actor-client	agent lifecycle + actions (`increment` → count)
02 agent-loop	autonomous `@loop` start/exit events (`group_id`)
03 agent-agent	**cross-agent calls** (caller → callee.action)

e.g. 03 shows the chain `DEADBEEF` → `deadbeef` → `feebdaed` across three agents.

How to inspect the provenance

Three built-in ways, for *any* run — all terminal (from an example folder):

```
# 1) tailored, content-aware analysis + text dashboard (printed by each run)
python solution.py                          # STEPs 2-4 print the report

# 2) analyze any captured buffer / whole run dir
python ../../../provenance/analyze.py runs/03-agent-agent_*

# 3) interactive queries: ask("...") turns NL into pandas
python ../../../provenance/query.py runs/03-agent-agent_* --ask "show the
    cross-agent calls"
```

The analysis auto-adapts: it reports only what the example produced (actions, loops, cross-agent calls, LLM calls, ...) — and any failures. Flowcept's markdown provenance card is written into each run dir.

The provenance dashboard — in the terminal

No images: each run prints an ASCII dashboard (records by subtype; LLM tokens by agent, or tasks by activity; capture overhead). Real `06-llm` run:

```
============================================================
06-llm -- provenance
============================================================
  records by subtype:
    academy_lifecycle  | ################################ 4
    llm_call           | ################                 2
    academy_action     | ################                 2
  LLM tokens by agent:
    Orchestrator       | ################################ 140
  capture overhead (total ms by category):
    flush              | ################################ 1.135
    llm_hook           | ########################         0.892
    intercept_task     | ######################           0.836
============================================================
```

Advanced — agents & LLMs

LLM backend: Argo → vLLM → OpenAI → local CPU model

- `ARGO_USER` set → ANL Argo gateway (on Aurora / ANL network), native tool calling
- else `VLLM_BASE_URL`/`OPENAI_BASE_URL` set → vLLM server we run (`VLLM_MODEL`); native tool calling when launched with `--enable-auto-tool-choice` (on Aurora: `source ../vllm_serve.sh && vllm_start`)
- else `OPENAI_API_KEY` set → OpenAI (`api.openai.com`, `OPENAI_MODEL`, default `gpt-4o-mini`), native tool calling
- else → local CPU model (`Qwen2.5-0.5B-Instruct`, offline, no GPU)
- **no mock**: every response is from a real model; `chat()` raises rather than fabricate text

Checked in priority order, first match wins; force with `FLOWCEPT_TUTORIAL_LLM=argo|vllm|openai|local`. Every call is recorded as an `llm_call` (model, prompt, response, tokens), linked to its action.

Examples 6 & 8: LLM-driven agents (vendored upstream)

```
# 06: the upstream Orchestrator builds a LangChain ReACT agent over a
# tool that messages a separate MySimAgent -- the LLM decides to call it.
agent = create_agent(make_chat_model(), tools=[sim_tool])
result = await agent.ainvoke({"messages": [("user", question)]})
```

- **06 llm**: ReACT agent → `llm_call` + `langgraph_node/graph`, and a cross-agent tool call when the LLM invokes the tool
- **08 discussion**: multi-agent group chat — three `GroupChatAgents` (Manager, Assistant, Senior Engineer) + `RoundRobinGroupChatManager` with a supervisor `@loop`; per-turn `llm_calls`, tokens per agent, bounded by `max_rounds`

The LLM is injected via `make_chat_model()` (Argo → vLLM → OpenAI → local) exactly where upstream builds `ChatOpenAI` — zero other agent-code changes.

What gets captured — a real `06-llm` run

The upstream ReAct agent, run on the **local 0.5B** model. One `@action` wraps one `langgraph_graph/node`, which drives one `llm_call` — all sharing the campaign:

```
* Records (subtype  activity)
    academy_action    answer              <- Orchestrator.@action
    langgraph_graph   LangGraph           <- create_agent ReACT graph
    langgraph_node    model               <- the LLM step in the graph
    llm_call          _AsyncChatHuggingFace
        prompt : ...simulated ionization energy of benzene...
        answer : The simulated ionization energy of benzene can be
                 calculated using quantum chemistry methods, such as DFT...
```

The action, the graph, its node, and the LLM call are one connected, queryable lineage. Note: the 0.5B model answered *directly* instead of calling the tool — provenance is exactly how you *see* that it skipped the tool. A tool-capable backend (Argo) instead emits a cross-agent `tool_call` to `MySimAgent`.

Extra advanced — failures

Example 7: cross-framework (Academy + LangGraph + real xTB)

The upstream `mol_design_agents.py`, unchanged: a LangGraph `StateGraph` (plan → tool_calling → simulate → conclude → critique → update, looping) runs inside an Academy `@loop`, and `simulate` calls **real GFN2-xTB** (RDKit → ASE → xtb).

- **one shared** `campaign_id` across both plugins (Academy + LangGraph)
- three layers in one lineage: `academy_loop` → `langgraph_node` → `tool_call` (the xTB energy)
- LLM = Argo `gpt-4o` (tool-capable) — drives the `tool_calling` node

Real run — `tool_call`=10, `langgraph_node`=9, `llm_call`=6, `academy_loop`=2 (1 campaign):

```
* nodes: plan, tool_calling(2), simulate(2), should_continue,
         conclude, critique, update
* real xTB ionization energies (charged - neutral, eV):
    simulate -> 13.1975...     simulate -> 14.1156...
```

Example 5: Parsl

```
class SimulationAgent(Agent):
    @action
    async def run_expensive_task(self) -> int:
        return await asyncio.wrap_future(expensive_task())  # a Parsl app
```

The agent delegates to a **Parsl** task; provenance captures the dispatching action and its result. (Example 07 shows the same pattern with real chemistry — the agent's process pool runs GFN2-xTB, each emitting a `tool_call`.)

Inspecting failures — where provenance shines

Agents (and LLMs) get things wrong. In **example 07 this is not staged** — the LLM proposes candidate molecules and some simply can't be built by RDKit/xTB. Flowcept records each failed `tool_call` with `status=ERROR` + real `stderr`:

```
status:  FINISHED 22   ERROR 10
!  Failures captured (status=ERROR) -- what the agent actually tried:
  tool_call/compute_ionization_energy  [ERROR]
    stderr: Parse failure for C3N2O
  tool_call/compute_ionization_energy  [ERROR]
    stderr: Parse failure for CF3NC(N)=O
  tool_call/compute_ionization_energy  [ERROR]
    stderr: Parse failure for C6H5NC(N)=O
  ... (8 xTB parse failures + 1 propagated langgraph_node)
```

You see *which* molecule the LLM invented, that xTB rejected it, and that successful runs still gave real energies (13.20, 14.12 eV) — the debugging payoff.

Running on Aurora (offline, terminal-only)

Each `exercises/aurora/<id>/` has its own `submit.pbs` (`-A ATPESC2026`); all share `env.sh` (modules + conda + offline LLM).

```
export FLOWCEPT_ENV_PREFIX=/lus/flare/projects/ATPESC2026/prov/envs/flowcept-
    academy
bash setup/install.sh aurora      # frameworks base + delta, no LLM download
source exercises/aurora/env.sh    # modules + conda --stack + HF offline
cd exercises/aurora/06-llm && qsub submit.pbs    # submit.pbs starts vLLM on
    the node
```

`install.sh aurora` adds a small delta on the `frameworks` module (nothing downloaded); all LLM usage reads ALCF's staged hub offline (`HF_HOME=/flare/datasets/model-weights`, `HF_HUB_OFFLINE=1`). **vLLM is the only backend** — each `submit.pbs` starts it on the node (no CPU fallback; `FLOWCEPT_USE_VLLM=0` → Argo). Details next slide.

vLLM on Aurora: serve it yourself (+ interactive)

On Aurora's XPU, `vllm serve` normally SIGSEGVs inspecting the model architecture. `vllm_start` dodges it: it **primes the modelinfo cache in-process** (model *class* only — no weights, no subprocess), then serves against the cache. Automatic, nothing to do by hand.

- **Adopt-on-source**: sourcing `vllm_serve.sh` in a fresh shell finds a running server + re-exports `VLLM_BASE_URL` (so `query.py --ask` routes to it).
- **Context**: `VLLM_MAX_MODEL_LEN=32768` (8192 overflowed — 08's group chat accumulates the whole discussion).

```
qsub -I -A ATPESC2026 -q debug -l select=1 -l walltime=60:00   # interactive
    node
source ../env.sh                                  # modules + conda + offline LLM
source ../vllm_serve.sh && vllm_start             # primes cache, serves on the GPUs
python solution.py                                # or: query.py runs/<id>_* --ask
    "..."
vllm_stop                                         # tear the server down; then exit
```

Recap

- **Capture:** one line (`captured()`) instruments any Academy example — zero agent-code changes.
- **Examples:** actor, loop, cross-agent, multi-process, Parsl, LLM agents, and cross-framework LangGraph + real xTB (07).
- **LLM:** Argo → vLLM → OpenAI → local CPU model.
- **Inspect:** tailored analysis, text dashboards, provenance cards — all terminal, including failures (status=ERROR + stderr).
- **Outputs:** every run in its own `runs/<id>_<date-time>/`.

instrument → capture → inspect (even the failures)

Thank you!

`exercises/{local,aurora}` · `provenance/analyze.py` · `slides/` · **upstream examples:** `github.com/academy-agents/academy`